

Soft start | Punkte: 2

Let's create a folder structure and some files that we can work with in the following exercises. Log in to your Linux machine and open a terminal by pressing  Textbox 1 .

Type pwd and press Enter. What does the terminal show you?  Textbox 2

Textbox 1

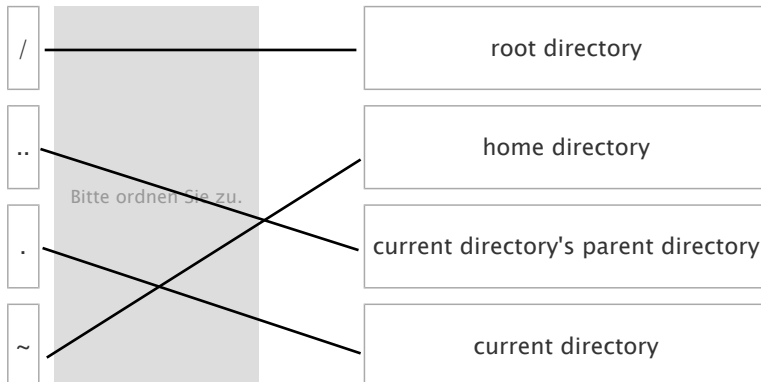
- ☒ Ctrl + Alt + T
- ☐ Ctrl + S
- ☐ Ctrl + Shift + C

Textbox 2

- ☐ my password
- ☒ the current working directory
- ☐ the content of my clipboard

Path description | Punkte: 2

Match the path description elements to their meaning.



Moving around folders | Punkte: 4

Create a new directory called "remote" in your home directory (pwd is ~/) using a relative path.

```
mkdir remote
```

Go to the new folder from your home directory using a relative path.

```
cd remote
```

Imagine you are logged in as user "boldlemming". Remember the folder structure is as follows:

root

- home
 - boldlemming (this is boldlemming's home directory)
 - remote

Go to the root folder (the root of the whole file structure) using a relative path.

```
cd ../../..
```

Return to the home directory using an absolute path.

```
cd /home/boldlemming
```



Create files and folders

Punkte: 6

Create a folder named "food" in your current working directory using relative paths.

```
mkdir food
```

You want to check if the folder is there. Print the names of the contents of the current directory to the standard output.

```
ls
```

There might be hidden files. Print the names of the contents of the current directory to the standard output including them.

```
ls -a
```

Create a new empty file named "lemmings_rumour.txt" in the current working directory using relative paths.

```
touch lemmings_rumour.txt
```

Create the 2 directories "route" and "coast" in parallel in the current working directory using only one command and relative paths.

```
mkdir route coast
```

Create the folder "company" in the current working directory and inside it the folder "family" using one command and relative paths.

```
mkdir -p company/family
```

Useful commands | Punkte: 5

What is the command to remove old commands and outputs from the terminal window?  Textbox 1

What is the command to see all previously executed commands?  Textbox 2

What is the command to see the manual for a command?  Textbox 3

How to you exit the manual for a command? Pressing  Textbox 4

What is the command to execute the following command (in the same line!) as the super user?  Textbox 5

Textbox 1

- ☒ clear
- ☐ clr
- ☐ clt

Textbox 2

- ☒ history
- ☐ prev
- ☐ cmds

Textbox 3

- ☒ man
- ☐ ttrl
- ☐ instr

Textbox 4

- ☒ q
- ☐ e
- ☐ x

Textbox 5

- ☒ sudo
- ☐ root
- ☐ rootexec

Redirection | Punkte: 3

This exercise only uses relative paths. Please forward the manpage entry of the command whoami into the new file "whoami_manual.txt" using redirection.

```
man whoami > whoami_manual.txt
```

Append the string "This is the manual file, status 16.10.2020." to the end of "whoami_manual.txt" using redirection again. Remember that you need a command that echoes its input so that you can redirect its output.

```
echo "This is the manual file, status 16.10.2020." >> whoami_manual.txt
```

Open "whoami_manual.txt" in the nano Editor.

```
nano whoami_manual.txt
```

Copy and move | Punkte: 5

Imagine the following folder structure:

```
~  
- food (empty)  
- route (has files)  
- coast  
  - jump_points (has files)  
  - properties (has files)  
- company  
  - family (empty)  
lemmings_rumour.txt  
... some other files...
```

This exercise uses only relative paths. Your pwd is ~. Copy "lemmings_rumour.txt" into the folder "route".

```
cp lemmings rumour.txt route
```

Copy the folder "route" into the folder "coast".

```
cp -r route coast
```

Rename "lemmings_rumour.txt" to "pure_truth_about_lemmings.txt".

```
mv lemmings rumour.txt pure truth about lemmings.txt
```

Rename the folder "company" into "hiking_friends".

```
mv company hiking friends
```

Move the file "pure_truth_about_lemmings.txt" to one folder above.

```
mv pure truth about lemmings.txt ..
```

Permissions | Punkte: 6

This exercise uses relative paths only.

Print the contents of the current working directory to the standard output such that the file permissions are shown.

```
ls -l
```

Imagine the output is the following:

```
- rwxrwxrwx 1 anxiousgiraffe admins    0  Oct 20 15:39 contacts.vcf
- rwxrwxrwx 1 anxiousgiraffe admins 4300 Dec 11 20:04 calendar.xlsx
```

Imagine the user that you are logged in as has all the necessary privileges so you don't need to worry about using sudo. Change the user-owner of "contacts.vcf" to "boldlemming".

```
chown boldlemming contacts.vcf
```

Change the group-owner of "calendar.xlsx" to "aliasgroup".

```
chgrp aliasgroup calendar.xlsx
```

Make "calendar.xlsx" read-only for world/other using the absolute mode. Leave the other permissions unchanged.

```
chmod 774 calendar.xlsx
```

Make "calendar.xlsx" free to read and write, but do not allow to execute for the group using symbolic mode and the "=" specifier.

```
chmod g=rw- calendar.xlsx
```

Add the execute permission for the world/other to "calendar.xlsx" back again using symbolic mode and the "+" specifier.

```
chmod o+x calendar.xlsx
```

The less command | Punkte: 1

less lets you...

- ☐ remove the last permission (order: rwx), supports specification of user/group/other
- ☒ show a file on the standard output, supports page stepping
- ☐ show the first few lines of a file on the standard output

 Sie dürfen maximal 1 Option(en) wählen.

**Specifying searches** | Punkte: 6

This exercise uses only relative paths. The command `grep` can be used to globally search for a regular expression and print matching lines. Search for all lines containing the string "parallel" in all files in the directory.

```
grep "parallel" *
```

Search for all lines containing the string "parallel" in all files in the directory and its subdirectories.

```
grep -r "parallel"
```

Search for all lines containing the string "parallel" in all text (.txt) files in the directory.

```
grep "parallel" *.txt
```

Search for all lines containing the string "parallel" in all files in the directory. Make sure that lines containing "PARALLEL" are also found.

```
grep -i "parallel" *
```

Count all lines containing the word "parallel" in all files in the directory without using a pipe. "parallelization" should not be found.

```
grep -cw "parallel" *
```

Combining files | Punkte: 2

The file "results.dat" contains simulation results, one per line. You are sent an update results_1.dat that contains new results for the same simulation but not the previous ones. Print the two files on the terminal with one command to look at them together.

```
cat results.dat results_1.dat
```

After checking them you decide to save all results in one place. Combine the two files into the new file "results_final.dat".

```
cat results.dat results_1.dat > results_final.dat
```

**Environment variables** | Punkte: 7

Print the name of your computer to the standard output using an environment variable and the echo command.

```
echo $HOSTNAME
```

Assign the name of your computer from the environment variable to a new variable called "MYCOMPUTER".

```
MYCOMPUTER=$HOSTNAME
```

Print the value of this new variable to the standard output.

```
echo $MYCOMPUTER
```

Delete the new variable again.

```
unset MYCOMPUTER
```

Redirect your user ID from the respective environment variable into a new file called "user_info.txt".

```
echo $UID > user_info.txt
```

Add your username by appending the value of the respective environment variable to "user_info.txt".

```
echo $USER >> user_info.txt
```

Navigate to your home directory via an absolute path using the value of the respective environment variable.

```
cd $HOME
```

Counting | Punkte: 4

This exercise uses relative paths only. You've been writing a small report for an exercise in nano, but you miss the GUI functionality to count the words as you have a restriction on that. Use the Linux command line to count the words in the file "intermediate_report.txt".

```
wc -w intermediate_report.txt
```

To check whether it will fit into a nicely formatted template, count the lines in the file.

```
wc -l intermediate_report.txt
```

Show the size of the file in bytes without using the ls command to check whether it is small enough to attach it to an email.

```
wc -c intermediate_report.txt
```

You notice that you used American English and British English inconsistently. Print all lines containing "analyse" case insensitively with their line numbers so that you can find and correct them later.

```
grep -ni "analyse" intermediate_report.txt
```

Wildcards | Punkte: 3

This exercise only uses relative paths. Your current working directory contains the following files:

report_1.pdf, report_2.pdf, report_2_draft.pdf, report_3_to_send_lemming.pdf, report_4.pdf, paper_draft.pdf, code.txt, mode.txt, node.txt, test.txt.

Use a single character wildcard with the ls command to show only the report pdfs that are numbered properly (i.e. that contain no "draft" or "to_send_lemming").

```
ls report ?.pdf
```

Use a multiple character wildcard with the ls command to show all pdfs.

```
ls *.pdf
```

Use a wildcard that chooses from a set of characters with the ls command to show code.txt, mode.txt and node.txt.

```
ls [mnc]ode.txt
```

Deleting with care | Punkte: 5

Imagine the following folder structure:

~

- food (empty)
- route (has files)
- coast
 - jump_points (has files)
 - properties (has files)
- company
 - family (empty)

lemmings_rumour.txt

... some other files...

Delete the file "lemmings_rumour.txt".

```
rm lemmings_rumour.txt
```

Delete the folder "food" using a command specifically intended for directories.

```
rmdir food
```

Delete the folder "company" with the folder "family" inside it using a command specifically intended for directories.

```
rmdir -p company/family
```

Delete the folder "cliff" recursively. Remember that it is not empty.

```
rm -r cliff
```

You are not quite sure which of the remaining elements you want to keep. Delete them interactively (have the command ask you for every file separately).

```
rm -i *
```

Analyzing a log file | Punkte: 3

This exercise uses only relative paths. A log file named "operations.log" is structured as follows:

[YYYY-MM-DD HH:mm:ss] [user] [descriptive text]

Print only the [user]-field from the log file using awk.

```
awk '{print $3}' operations.log
```

Print only the [user]-field from the log file from entries that were written on Oct. 20th 2020 using grep and awk.

```
grep "2020-10-20" operations.log | awk '{print $3}'
```

Sort the users in the log file that have entries on Oct. 20th 2020 alphabetically and print each user only once using sort, uniq, grep, and awk.

```
grep "2020-10-20" operations.log | awk '{print $3}' | sort | uniq
```

Specifying searches (2) | Punkte: 3

This exercise uses relative paths only. A log file named "operations.log" is structured as follows:

YYYY-MM-DD/HH:mm:ss/user/descriptive text

Print all lines excluding information about login and logout of users. The unique property of login /logout entries is the word "Authentication" in the description. No need for case insensitive search. Note that you need the whole line.

```
grep -v "Authentication" operations.log
```

Extract the user and description. Sort this information by user and display the names starting from "z".

```
grep -v "Authentication" operations.log | awk -F/ '{print $3,$4}' | sort -r
```

Imagine you have several of these log files in a folder. Print the names of all files that contain entries for the user "boldlemming" (no need to specify that it has to appear in the user field). Search only files with .log file extension.

```
grep -l "boldlemming" *.log
```